

STEP-BY-STEP PROJECT
BLUEPRINT

Building an Agentic GenAI Data Scientist Portfolio Project

A concise, end-to-end guide to designing, building, and demoing a production-style multi-agent GenAI + RAG system — mapped directly to a real Data Scientist / GenAI Consultant job description.

Reference stack: Dataiku LLM Mesh + Snowflake Cortex • Works equally well on any GenAI platform — LangChain, LlamaIndex, OpenAI/Anthropic APIs, Postgres, BigQuery, Pinecone, Chroma, and more.

Includes a ready-to-use synthetic dataset — 7 linked tables, 3,700+ records, structured + unstructured — so you can start building today.

How to use this guide

This guide turns a real job description into a project you can actually build, demo, and talk through in an interview. It is intentionally **platform-agnostic**: every step lists the Dataiku + Snowflake reference stack named in the job post *and* an equally valid open alternative. Dataiku and Snowflake experience is a nice-to-have here, not a requirement — pick whichever stack you already know, or use this as a reason to try something new.

What you get alongside this guide

A synthetic, fully fictional dataset ("PharmaSense AI") — 7 linked CSV/Excel tables covering compounds, clinical trials, sites, lab results, adverse events, an unstructured research-document corpus for RAG, and simulated agent usage logs for building evaluation dashboards.

Provided as both a formatted Excel workbook (with a full data dictionary) and a plain-CSV .zip for loading straight into Snowflake, Postgres, BigQuery, DuckDB, or pandas.

The role, decoded

Every requirement in the job post maps to something concrete you will build in this project:

Job description asks for...	You demonstrate it in...
Design & deploy production-ready GenAI / Agentic AI workflows	Steps 4–8 — the full agent build, orchestration, and deployment
Build AI Agents; write multi-page agent instructions; define tools	Step 4 — written agent briefs + tool specs for 5 specialist agents
Architect multi-agent orchestration for end-to-end task execution	Step 6 — Router/Planner pattern with sequential + parallel hand-offs
Dataiku (LLM Mesh) integrated with Snowflake	Steps 2, 3, 7 — reference stack (optional — any platform accepted)
LLMs, prompt engineering, RAG, vector databases	Step 3 — RAG layer over the research-document corpus
Evaluate tech choices, architecture, trade-offs	Step 9 — an architecture & trade-offs one-pager
Walk through the project end-to-end	Step 10 — your interview narrative & demo script
Pharma / Life Sciences exposure (nice-to-have)	The whole scenario — R&D; + clinical ops use case

1 · The Project You'll Build: PharmaSense AI

PharmaSense AI is a multi-agent GenAI assistant for a fictional pharma R&D; and clinical operations team. Scientists, clinical ops managers, and regulatory staff ask natural-language questions — the system routes each question to the right specialist agent, grounds its answer in real (synthetic) company data, and can autonomously triage and escalate safety signals.

Example questions the finished system should answer

- “Which Phase II oncology trials are below 60% enrollment right now?” (*structured data + SQL*)
- “What has our internal research said about JAK2 inhibitors and cardiotoxicity?” (*RAG over documents*)
- “A site just reported a serious adverse event for Trial TRL-0032 — triage it.” (*classification + escalation tool*)
- “Give me the full picture on compound DKU-1042: labs, trials, safety, and related literature.” (*parallel multi-agent fan-out + report synthesis*)

Architecture at a glance

Layer	What it does	Reference stack	Any-platform alternative
Application Layer	Chat UI / API endpoint students interact with	Dataiku Agent Hub app	FastAPI + Streamlit / Gradio
Orchestration Layer	Router agent classifies intent; fans out to specialists; Report Writer merges results	Dataiku Agent Hub orchestration	LangGraph / CrewAI / AutoGen / hand-rolled
Agent + Tool Layer	5 specialist agents, each with its own instructions + tools	Dataiku LLM Mesh tool calling	LangChain / OpenAI / Anthropic function calling
Retrieval Layer	Embeddings + vector search over research_documents	Snowflake Cortex Search	sentence-transformers + FAISS / Chroma / pgvector
Data Layer	7 structured + unstructured tables (provided)	Snowflake	Postgres / BigQuery / DuckDB / pandas

Note the pattern, not the product: whichever tools you pick, keep these five layers distinct. That separation is exactly what an interviewer means by “architecture and trade-offs.”

2 · Step-by-Step Build Guide

STEP 1 Set up your data foundation

Goal — Stand up structured + unstructured data so your agents have something real to reason over.

- Load the 7 provided tables (compounds, clinical_trials, trial_sites, lab_results, adverse_events, research_documents, agent_interaction_logs) into a database.
- Keep **research_documents.full_text** aside — it is your unstructured corpus for Step 3.
- Sanity-check referential integrity (e.g. every trial_id in adverse_events exists in clinical_trials).

	Reference stack (Dataiku + Snowflake)	Any-platform alternative (fully valid)
Tools	Snowflake: create a database, <i>COPY INTO</i> from the CSVs (or Snowsight UI upload); connect it as a Dataiku dataset.	Postgres, BigQuery, DuckDB, or SQLite — or skip a database entirely and load the CSVs with pandas for a lighter-weight build.

Output / artifact

A queryable structured schema (7 linked tables) plus a raw text corpus, ready for the rest of the project.

STEP 2 Choose & wire up your GenAI platform

Goal — Decide where your LLM calls, prompts, and governance will live — one governed layer every agent shares.

- Add at least one LLM connection (hosted API or self-hosted model).
- Wrap it behind a single internal function, e.g. `call_llm(prompt, model, guardrails)`, so every agent calls the same governed entry point.
- Turn on basic usage/cost logging from day one — you'll reuse it in Step 7.

	Reference stack (Dataiku + Snowflake)	Any-platform alternative (fully valid)
Tools	Dataiku LLM Mesh: add an LLM connection (OpenAI / Anthropic / Azure / self-hosted), which centralizes API keys, cost tracking, and guardrails automatically.	A thin wrapper around the OpenAI/Anthropic/Bedrock SDK, or a LangChain/LlamaIndex model-router class — the pattern matters more than the product.

Output / artifact

A single, swappable LLM access function that every agent in the system calls.

STEP 3 Build the RAG layer over your documents

Goal — Ground answers in real company text instead of letting the model hallucinate.

- Chunk each `full_text` value (roughly 300–500 tokens per chunk).
- Embed every chunk and index it in a vector store.
- Wrap retrieval in a single tool: `vector_search_tool(query, k)` that returns ranked, citable passages.

	Reference stack (Dataiku + Snowflake)	Any-platform alternative (fully valid)
Tools	Snowflake Cortex Search — managed hybrid vector + keyword search with automatic embeddings (Arctic Embed), called directly from the LLM Mesh.	sentence-transformers embeddings indexed in FAISS, Chroma, pgvector, or OpenSearch, queried through LangChain or LlamaIndex.

Output / artifact

A working retrieval tool that returns grounded, cited passages for any free-text question.

STEP 4

Design your agents & write their instructions

Goal — Give each agent a clear, multi-page brief — exactly as the job post asks for.

- Draft one instructions document per agent (roughly 500–1,500 words): identity & scope, available tools and when to use each one, output format, escalation/refusal rules, worked examples.
- Define the 5 specialist agents: Trial Data Analyst, Literature & Document Research, Adverse Event Triage, Compound Similarity, and Report Writer.
- Write a JSON tool spec (name, description, parameters) for every tool each agent may call.

	Reference stack (Dataiku + Snowflake)	Any-platform alternative (fully valid)
Tools	Dataiku Agent Hub agent definitions, with tools wired to LLM Mesh connections.	Plain markdown instruction files + OpenAI/Anthropic function-calling tool schemas — framework-optional.

Output / artifact

One instructions.md per agent, plus a tool-spec file every agent can be tested against.

STEP 5

Implement each agent's tools

Goal — Turn every tool spec from Step 4 into working, testable code.

- Build a read-only, parameterized `sql_query_tool` against your warehouse.
- Wire in the `vector_search_tool` from Step 3.
- Add `ae_severity_classifier_tool`, `compound_similarity_tool`, `citation_formatter_tool`, and a simulated `escalation_notifier_tool` (a log line or webhook is enough).
- Unit-test every tool against a handful of known rows before wiring it into an agent.

	Reference stack (Dataiku + Snowflake)	Any-platform alternative (fully valid)
Tools	Dataiku Agent Hub tool definitions (Python recipes exposed as callable tools), or Snowflake Cortex Agents' tool configuration.	Plain Python functions registered as LangChain tools or OpenAI/Anthropic function-calling definitions.

Output / artifact

A tested tool library with clear input/output schemas that any agent can call.

STEP 6

Orchestrate the multi-agent system

Goal — Route a natural-language request to the right specialist(s) and merge their outputs into one answer.

- Build a Router / Planner agent that classifies user intent.
- Demonstrate three orchestration patterns: single-agent tool use, sequential hand-off (Analyst → Report Writer), and parallel fan-out/fan-in (Analyst + Literature Research run together, then merge).
- Log every step of the flow — you already have a template for this in `agent_interaction_logs.csv`.

	Reference stack (Dataiku + Snowflake)	Any-platform alternative (fully valid)
Tools	Dataiku Agent Hub orchestration, or Snowflake Cortex Agents' built-in orchestration.	LangGraph, CrewAI, AutoGen, or a hand-rolled state machine — a simple one is easier to explain clearly in an interview than a black-box framework.

Output / artifact

A working end-to-end flow: user question → one or more agents → one final, cited answer.

STEP 7

Add guardrails, evaluation & observability

Goal — Make the system trustworthy and measurable — this is what separates a demo from something production-credible.

- Guardrails: redact patient_code-style identifiers, screen retrieved text for prompt-injection patterns, and add refusal rules for out-of-scope medical advice.
- Evaluation: build a 20–30 question “golden set” spanning SQL, RAG, and AE-triage questions; score faithfulness, relevance, latency, and cost per run.
- Observability: log model, tokens, latency, tool used, and escalations for every interaction.

	Reference stack (Dataiku + Snowflake)	Any-platform alternative (fully valid)
Tools	Dataiku LLM Guard Services for policy enforcement, plus Mesh's built-in cost/usage dashboards.	RAGAS or DeepEval for automated RAG scoring, plus a simple logging table — even a spreadsheet pivot table is a legitimate portfolio-project dashboard.

Output / artifact

A short evaluation report (pass rate, average latency/cost, flagged failures) and a one-page guardrails checklist.

STEP 8

Deploy & demo

Goal — Make the system usable by someone other than you — and easy to show off.

- Wrap the orchestrated system behind a chat interface or API endpoint.
- Record or rehearse a 2–3 minute demo that shows at least one question per specialist agent.

	Reference stack (Dataiku + Snowflake)	Any-platform alternative (fully valid)
Tools	Publish as a Dataiku Agent Hub app, or a Snowflake Cortex Agents API endpoint.	A FastAPI backend with a Streamlit or Gradio chat UI, containerized with Docker for a portable, easy-to-share demo.

Output / artifact

A live (or locally runnable) chat demo plus a short recorded walkthrough.

STEP 9

Document architecture & trade-offs

Goal — Prepare to “walk through the project end-to-end, evaluating tech choices, architecture, and trade-offs” — verbatim from the job post.

- Write a one-page architecture doc with the 5-layer diagram from Section 1.
- List 2–3 alternative approaches you considered and rejected, and why.
- Note current limitations and what you'd change at 10x the data volume or user load.

Platform-neutral step — Same one-pager regardless of stack — this step is deliberately platform-neutral.

Output / artifact

An ARCHITECTURE.md (or one-page PDF) ready to attach to your portfolio or resume.

STEP 10

Prepare your interview narrative

Goal — Be ready to speak fluently to every must-have skill in the job post, using this one project as evidence.

- For each must-have skill, write a 30–60 second talking point that references a specific decision you made while building this project (see the Appendix template).
- Rehearse a live walkthrough in this order: problem → data → architecture → live demo → results → what you'd improve next.

Platform-neutral step — Not a tooling step — it's the payoff step. This is what the interview process is actually testing.

Output / artifact

A one-page “talking points” cheat sheet you can review five minutes before any interview.

3 · Guardrails & Evaluation Checklist

Use this as a pre-demo checklist. None of these require Dataiku or Snowflake specifically — they are the practices an interviewer is really listening for.

Category	Checklist item
Grounding	Every RAG answer cites the source document(s) it used.
Grounding	The system says “I don't know” when retrieval returns nothing relevant, instead of guessing.
Safety	Patient/site identifiers are redacted or tokenized before reaching the LLM.
Safety	The Adverse Event Triage agent auto-escalates any “Serious” event to a human reviewer.
Robustness	Retrieved text is screened for prompt-injection patterns before being trusted as instructions.
Evaluation	A golden set of 20–30 questions is scored for faithfulness, relevance, and correctness.
Observability	Every agent call logs model, tokens, latency, tool used, and cost.
Cost	You can state an approximate cost-per-query and where you'd optimize it first.

4 · Appendix

A. Talking-points template

Fill one row per must-have skill before any interview. Keep each answer to one concrete decision, not a general description.

Must-have skill	The decision I made ...	Why (trade-off considered)
Dataiku / LLM Mesh (optional)		
Snowflake (optional)		
RAG & vector databases		
Agent tooling & orchestration		
Prompt engineering		

B. Glossary

Term	Meaning
Agentic AI	An LLM-based system that can decide which tools/steps to use to complete a task, rather than just answering in one shot.
RAG	Retrieval-Augmented Generation — grounding an LLM's answer in retrieved documents rather than its training data alone.

Term	Meaning
Vector database	A database optimized to store embeddings and retrieve the most semantically similar ones to a query.
LLM Mesh	A governed gateway layer between applications and one or more LLM providers, handling routing, cost, and guardrails.
Tool calling / function calling	A pattern where the LLM chooses to invoke a defined function (e.g. a SQL query) and uses its result.
Multi-agent orchestration	Coordinating several specialist agents (sequentially or in parallel) to complete one larger task.
Guardrails	Automated checks (safety, PII, injection, refusal rules) applied around LLM inputs/outputs.

C. Dataset quick reference

Provided alongside this guide as **pharmasense_synthetic_dataset.xlsx** (formatted workbook with a full data dictionary) and **pharmasense_synthetic_data_csv.zip** (plain CSVs for any database).

Table	Rows	Use it for
compounds	150	Compound master data — SQL agent, similarity tool
clinical_trials	~110	Trial status/enrollment questions
trial_sites	~375	Site-level enrollment & geography
lab_results	2,000	Preclinical assay trend questions
adverse_events	500	Adverse Event Triage agent
research_documents	250	RAG corpus (unstructured full_text)
agent_interaction_logs	400	Evaluation & observability dashboards

All data in this project is entirely synthetic and fictional — no real compounds, trials, patients, or individuals are represented. Built for educational / portfolio use.